

02443 Stochastic Simulation

Day 6: Simulated annealing

Nicolai Siim Larsen
Technical University of Denmark
DTU Compute
Section for Statistics and Data Analysis

Agenda

- 1 Introduction
- 2 The mathematics of simulated annealing
- 3 The algorithm
- 4 Examples
- 5 Exercise

Agenda

- 1 Introduction
- 2 The mathematics of simulated annealing
- 3 The algorithm
- 4 Examples
- 5 Exercise

Simulated Annealing

Simulated annealing is a stochastic optimization algorithm designed to solve large-scale optimization problems and avoid becoming trapped in local optima.

Basic idea: From optimization to sampling

We transform the optimization problem into a sampling problem by constructing a sequence of probability distributions whose most likely states are the optimal ones. Simulated annealing is an algorithm that samples from these distributions, with gradually decreasing temperatures, using MCMC methods.

Simulated annealing = Metropolis-Hastings + A temperature scheme.

Similar ideas as in MCMC

As in the Metropolis-Hastings algorithm, we do not need to know the optimal state in advance. In particular, we only need to compare objective values between states in order to determine which state is preferable.

A general optimization problem

Consider the optimization problem

$$f^{\star} = \min_{x \in S} f(x),$$

where

- S is the state space (the set of feasible values/states),
- $f : S \rightarrow \mathbb{R}$ is the objective function.

We seek a minimizer

$$x^{\star} = \operatorname{argmin}_{x \in S} f(x).$$

The minimizer is not necessarily unique. We therefore define the set of optimal solutions

$$M = \{x \in S : f(x) = f^{\star}\}.$$

We shall assume that the number of minimizing points is finite, i.e., $|M| < \infty$. This is typically the case for discrete optimization problems, where also S is finite, i.e., $|S| < \infty$.

Agenda

- 1 Introduction
- 2 The mathematics of simulated annealing
- 3 The algorithm
- 4 Examples
- 5 Exercise

Constructing a probability distribution

Our objective is to find states with small values of the objective function $f(x)$. To achieve this, we introduce the probability distribution

$$P_T(x) = \frac{e^{-f(x)/T}}{\sum_{y \in S} e^{-f(y)/T}}, \quad x \in S,$$

where $T > 0$ is called the *temperature*.

Observations

- States with small objective values receive high probability.
- States with large objective values receive low probability.
- The normalizing constant is generally difficult to calculate.

We can use MCMC methods to sample from a distribution with a density only known up to a multiplicative constant.

Why this distribution?

Recall the distribution

$$P_T(x) = \frac{e^{-f(x)/T}}{\sum_{y \in S} e^{-f(y)/T}}.$$

Factoring out $e^{-f^*/T}$ from the denominator gives

$$P_T(x) = \frac{e^{(f^* - f(x))/T}}{|M| + \sum_{y \in S \setminus M} e^{(f^* - f(y))/T}}.$$

As the temperature decreases:

- For optimal states $x \in M$: $f^* - f(x) = 0$.
- For non-optimal states $x \notin M$: $f^* - f(x) < 0$.

Consequently,

$$e^{(f^* - f(x))/T} \longrightarrow 0 \quad \text{for all } x \notin M,$$

as $T \rightarrow 0$.

Metropolis-Hastings for optimization

Goal: Sample from a distribution with density

$$P_T(x) \propto e^{-f(x)/T}.$$

Applying the Metropolis-Hastings algorithm with a symmetric proposal distribution, a transition from the current state x to a proposed state y is accepted with probability

$$\alpha(x, y) = \min \left(1, \frac{P_T(y)}{P_T(x)} \right) = \min \left(1, e^{-(f(y)-f(x))/T} \right).$$

Simulated annealing is simply Metropolis-Hastings applied to the distribution P_T .

Interpretation

- If $f(y) < f(x)$, the candidate is always accepted.
- If $f(y) > f(x)$, the candidate is accepted with a probability $e^{-(f(y)-f(x))/T}$.

Question: Why would you move to a worse state?

Agenda

- 1 Introduction
- 2 The mathematics of simulated annealing
- 3 The algorithm**
- 4 Examples
- 5 Exercise

The simulated annealing algorithm

Let

$$T_0 > T_1 > T_2 > \dots$$

be a sequence of decreasing temperatures.

At iteration i :

- 1 Generate a candidate state Y_i from the current state X_i .
- 2 If $f(Y_i) \leq f(X_i)$, accept the candidate.
- 3 If $f(X_i) \leq f(Y_i)$, accept the candidate with probability $\exp\left(-\frac{f(Y_i)-f(X_i)}{T_i}\right)$.
- 4 Set the temperature to the next value in the cooling scheme.

Key idea

The algorithm initially explores the state space aggressively and gradually becomes more selective as the temperature decreases.

Why lower the temperature?

Consider the acceptance probability

$$\alpha(X_i, Y_i) = \exp\left(-\frac{f(Y_i) - f(X_i)}{T_i}\right),$$

for a proposed state with $f(Y_i) > f(X_i)$.

High temperatures

$$\alpha(X_i, Y_i) = \exp\left(-\frac{f(Y_i) - f(X_i)}{T_i}\right) \approx 1.$$

The algorithm frequently accepts worse solutions and explores the state space.

Low temperatures

$$\alpha(X_i, Y_i) = \exp\left(-\frac{f(Y_i) - f(X_i)}{T_i}\right) \approx 0.$$

The algorithm rarely accepts worse solutions and concentrates on improving the objective value.

Practical considerations

The performance of simulated annealing depends strongly on several design choices.

Cooling schedule

- How quickly should the temperature decrease?
- Fast cooling may lead to poor solutions.
- Slow cooling increases the computational cost.

Initial solution

- Different initial states may lead to different trajectories.
- Multiple independent runs are often beneficial.

Practical considerations (2)

Candidate generation

- How should new candidate states be proposed?
- Small modifications improve local exploration.
- Large modifications improve global exploration.

Local refinement

- Simulated annealing is often combined with local search methods.
- The final solution can frequently be improved further using deterministic optimization techniques.

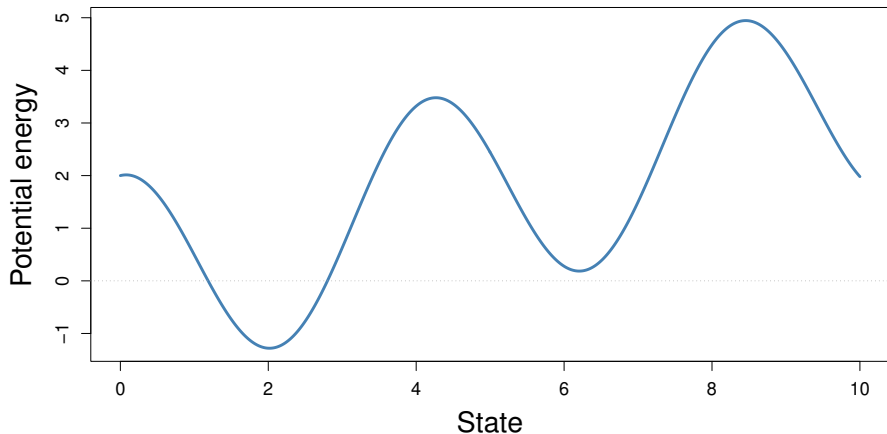
Agenda

- 1 Introduction
- 2 The mathematics of simulated annealing
- 3 The algorithm
- 4 Examples**
- 5 Exercise

An example energy landscape

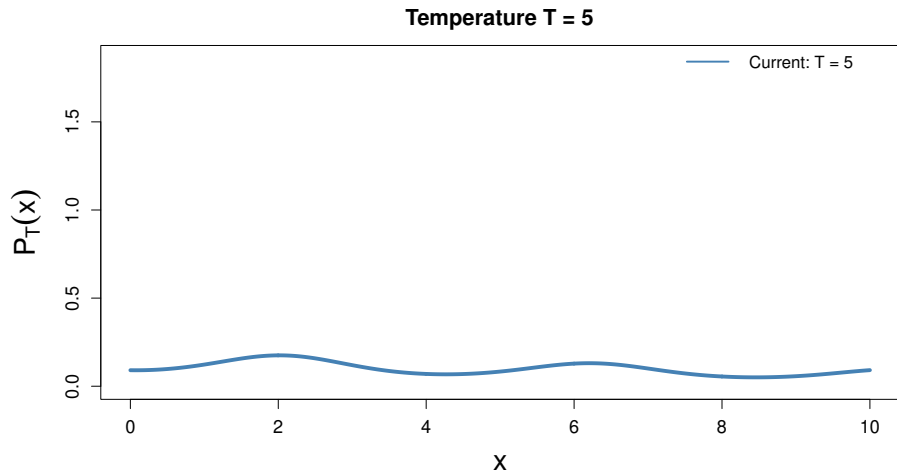
We apply simulated annealing to find the minimum of the potential energy function $U(x)$.

Potential Energy Landscape



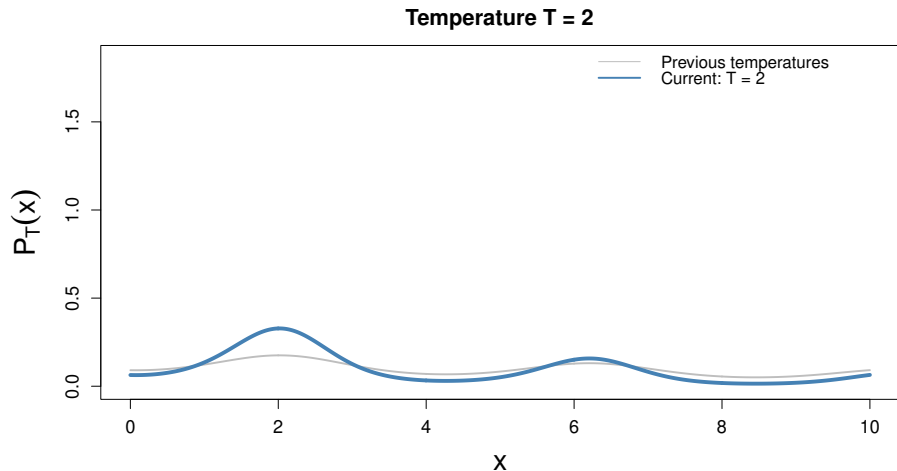
An example energy landscape - Distributions

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



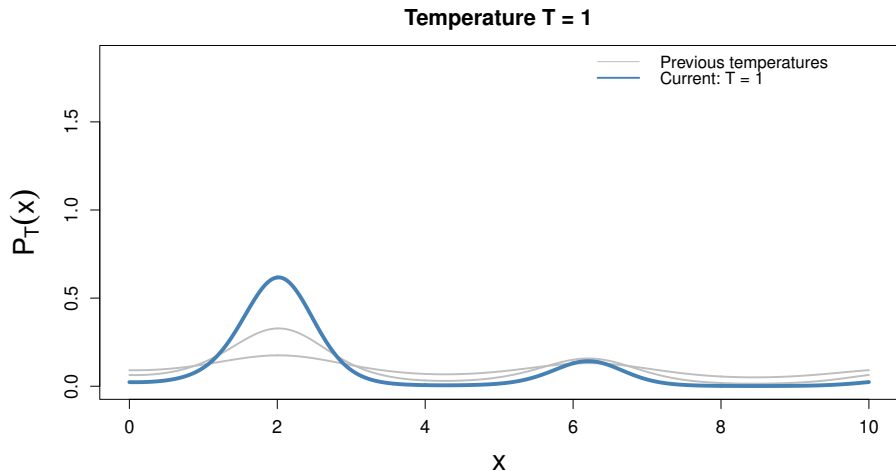
An example energy landscape - Distributions

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



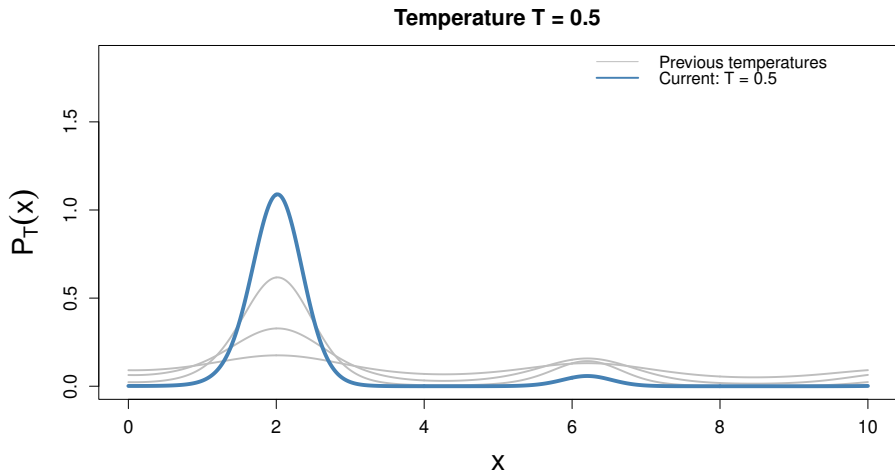
An example energy landscape - Distributions

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



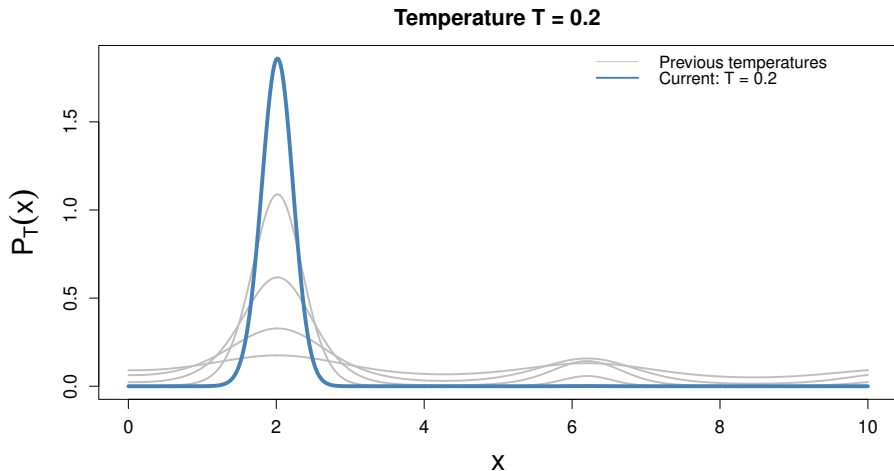
An example energy landscape - Distributions

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



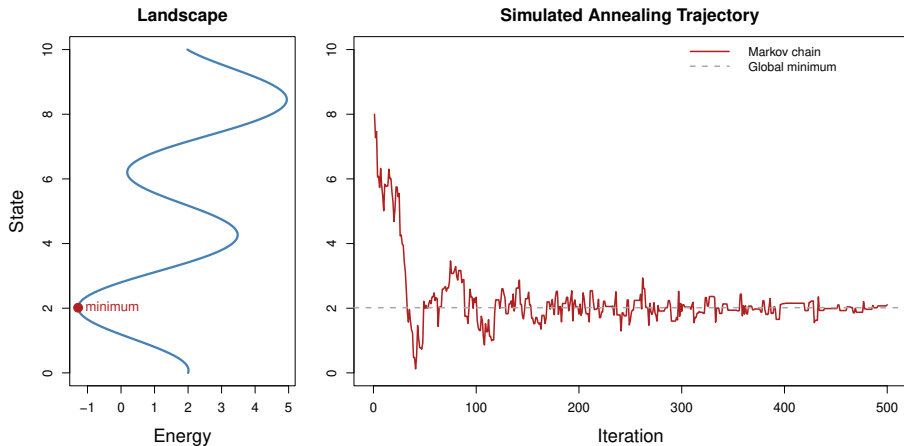
An example energy landscape - Distributions

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



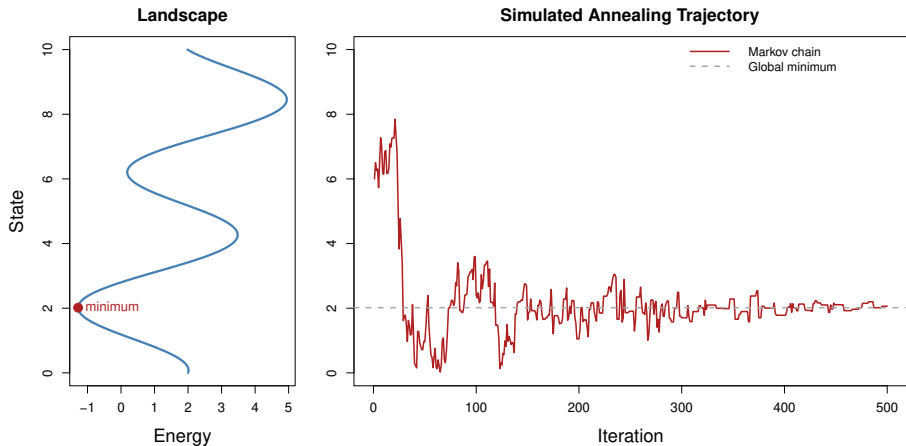
An example energy landscape - Runs

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



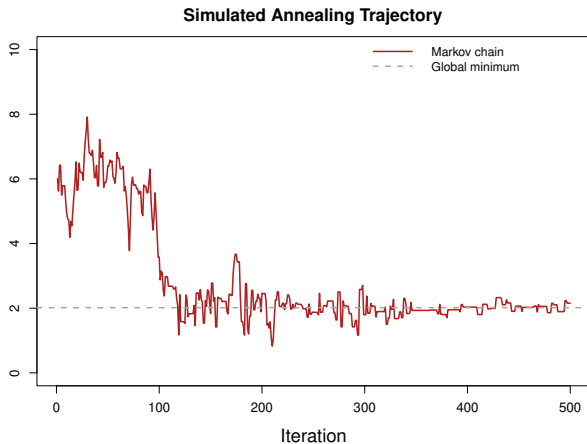
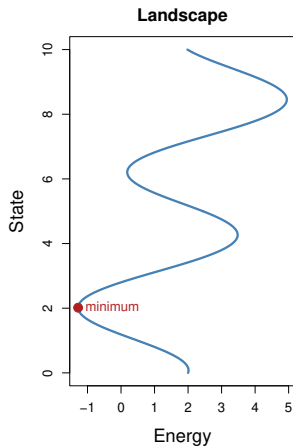
An example energy landscape - Runs

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



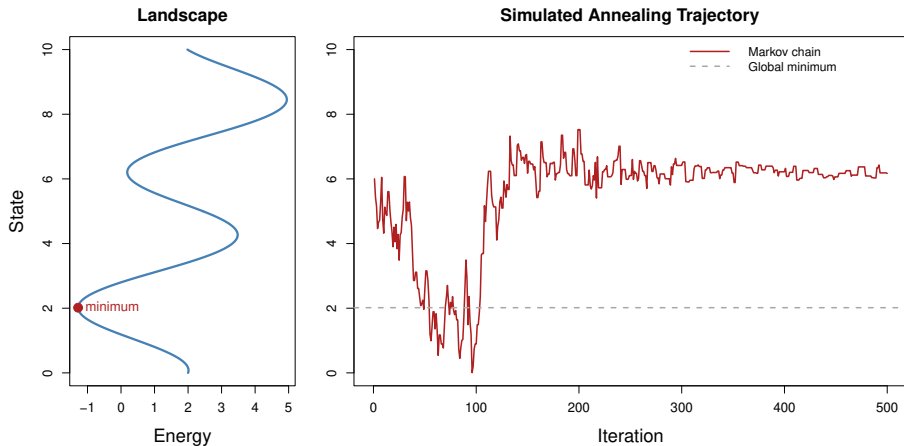
An example energy landscape - Runs

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



An example energy landscape - Runs

We apply simulated annealing to find the minimum of the potential energy function $U(x)$.



Travelling Salesman Problem (TSP)

The Travelling Salesman Problem (TSP) is a classical combinatorial optimization problem.

Given

- n cities,
- a cost matrix A ,

where A_{ij} denotes the cost of travelling from city i to city j .

We seek a route $S = (S_1, \dots, S_n)$ that

- starts and ends at the same city,
- visits every city exactly once,
- minimizes the total travel cost.

The objective function is

$$f(S) = \sum_{i=1}^{n-1} A_{S_i, S_{i+1}} + A_{S_n, S_1}.$$

Example: TSP Cost matrix

Consider the following travel-cost matrix:

Town from / to	1	2	3	4	5	6
1	–	5	3	1	4	12
2	2	–	22	11	13	30
3	6	8	–	13	12	5
4	33	9	5	–	60	17
5	1	15	6	10	–	14
6	24	6	8	9	40	–

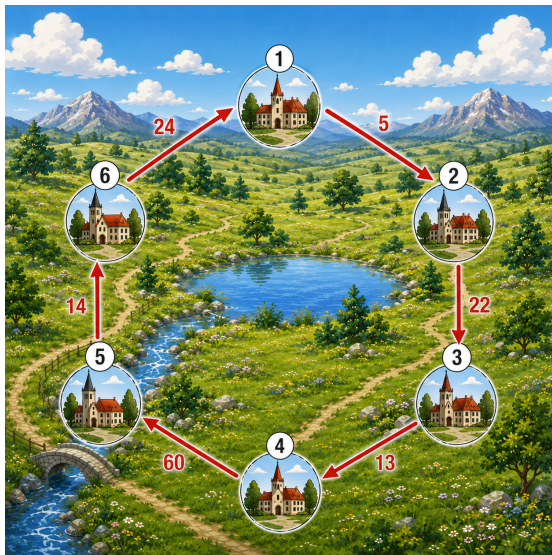
For the initial route

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 1,$$

the total cost is

$$5 + 22 + 13 + 60 + 14 + 24 = 138.$$

Example: TSP graph



Applying simulated annealing to the TSP

To apply simulated annealing, specify the components of the optimization problem.

State space

A state is a feasible route $S = (S_1, S_2, \dots, S_n)$ visiting every city exactly once and returning to the starting city.

Objective function

The objective value of a route is its total travel cost

$$f(S) = \sum_{i=1}^{n-1} A_{S_i, S_{i+1}} + A_{S_n, S_1}.$$

Candidate generation

Generate a new route by making a small modification to the current route, for example:

- swap two cities,
- reverse a segment of the route,
- move a city to a new position.

Acceptance rule

Accept the new route according to the simulated annealing acceptance probability.

Agenda

- 1 Introduction
- 2 The mathematics of simulated annealing
- 3 The algorithm
- 4 Examples
- 5 Exercise

Exercise 7

Implement the simulated annealing algorithm to solve the travelling salesman problem.

Part 1

- Select n points in the plane,
- Use the Euclidean distance between the points as the cost function.
- Generate proposals by swapping two randomly selected points on the route:

For $(1, 2, 3, 4, 5)$ propose e.g. $(1, 2, 5, 4, 3)$ or $(4, 2, 3, 1, 5)$

For $(3, 2, 5, 1, 4)$ propose e.g. $(4, 2, 5, 1, 3)$ or $(3, 5, 2, 1, 4)$

- Apply a cooling scheme, for example:

$$T_k = \frac{1}{\sqrt{1+k}}$$

$$T_k = \frac{1}{\ln(2+k)}$$

- Plot the resulting route in the plane. As a sanity check, test the algorithm using points located on a circle.

Exercise 7 (continued)

Part 2

- Modify the program to work with general travel costs.
- Apply the algorithm to the cost matrix found on Learn.
- Experiment with different design choices, such as cooling schedules and proposal mechanisms.